# EL-2003 – Computer Organization & Assembly Language Lab
# Lab # 5

**Objectives:**

- Apply conditional jumps to larger array problems, including range checks and multi-way decisions
- Choose between signed and unsigned conditional jumps based on the type of data being processed
- Use the shift instructions (SHL, SHR, SAR) and understand their effect on the carry flag and on signed and unsigned values
- Use the rotate instructions (ROL, ROR, RCL, RCR) and the carry flag to move and inspect individual bits
- Combine shifting with conditional jumps to examine and rearrange the bits of a byte

**Note: Carefully read the following instructions (*Each instruction contains a weightage*)**

1. First think about statement problems and then write your program.
2. Do not copy from any source otherwise you will be penalized with negative marks.
3. Add your source code in word document and attach output screenshots.
4. Upload word file with codes and screenshot.
5. Submit your lab on Google Classroom.

---

**Question 1:**

Write and execute an ASM program that defines ten unsigned words in memory using dw: 34, 87, 12, 99, 56, 73, 21, 64, 8, 45. Using BX as a pointer and CX as a counter, walk through the array exactly once. Initialise a register holding the smallest value so far to FFFFh, and update it whenever a smaller element is found. At the same time, count how many elements are strictly greater than 20 and strictly less than 80, so an element is counted only if it passes both tests. Use CMP with unsigned conditional jumps, and loop using SUB CX, 1 followed by JNZ. When the loop finishes, store the smallest value in a memory word labelled smallest and the count in a memory word labelled count. Expected results are smallest = 0008h and count = 0006h. Attach the code, a screenshot of CMD compiling the .asm file to .com, and an AFD screenshot showing both stored values in memory.

**Question 2:**

Write and execute an ASM program that performs the following three tasks. First, load AL with C6h five separate times, and on each fresh copy apply exactly one of SHL, SHR, SAR, ROL and ROR with a count of 1, storing each result into five consecutive bytes labelled res_shl, res_shr, res_sar, res_rol and res_ror. The expected values are 8Ch, 63h, E3h, 8Dh and 63h, and you must attach an AFD screenshot showing AL and the carry flag immediately after each of the five instructions. Second, set the carry flag using STC, load AL with 35h, execute RCL AL, 1 followed by RCR AL, 1, and store the final AL in a byte labelled res_rc. The expected value is 35h, and you should record the carry flag after each of the two instructions. Third, load AX with 13 and multiply it by 12 using only shift instructions and a single ADD

(no MUL instruction), storing the result in a word labelled product. The expected value is 009Ch. Terminate the program with MOV AX, 0x4C00 and INT 21h. Attach the code, the CMD compilation screenshot, and the AFD screenshots described above. In a short note, explain in one or two sentences why SAR and SHR give different results on C6h, and why the RCL then RCR pair in the second task returns the original value.

**Question 3:**

A weather station records eight temperature readings in degrees Celsius, stored as signed words: 22, -5, 31, 0, 14, -12, 35, 8. Write a program that classifies every reading as Freezing (below 0), Cold (0 to 14), Moderate (15 to 29) or Hot (30 and above), keeping a separate counter in memory for each category, and that also determines the highest and the lowest reading. Choose the appropriate family of conditional jumps for this kind of data, and explain in a short written note why using the unsigned family (JA, JB and their relatives) would have produced wrong results here, naming the specific readings that would have been misclassified and what category or value each would wrongly have received. Attach the code, a screenshot of CMD compiling the .asm file to .com, and AFD screenshots showing the four category counters and the highest and lowest values in memory.

**Question 4:**

A legacy sensor transmits each status byte with its bits in the reverse order from what your system expects, so its bit 0 is your bit 7, its bit 1 is your bit 6, and so on. Write a program that takes four status bytes stored in memory, B1h, A5h, 3Ch and 01h, and for each one produces the corrected byte (with the bit order reversed) using only shift and rotate instructions together with the carry flag, and stores it in a result array. In a second array, record 1 if the byte is a bit palindrome (identical before and after reversal) and 0 if it is not. The bytes must be processed in a loop using indirect addressing through a pointer or index register, and you must not use a lookup table or the MUL and DIV instructions. Attach the code, a screenshot of CMD compiling the .asm file to .com, and AFD screenshots showing the contents of both result arrays. In a short note, explain the role the carry flag played in moving each bit from the source byte into the result byte.